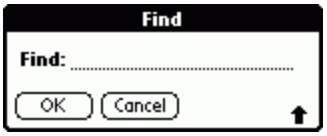
Palm OS Programming Recipes

Supporting Global Find



Any application that stores text data can benefit from supporting the Palm OS global find feature, which is launched when the user taps the **Find** icon in the Graffiti writing area.

To implement the global find feature, your application should:

1. Handle the `sysAppLaunchCmdSaveData` launch code, to save data if necessary.
2. Handle the `sysAppLaunchCmdFind` launch code, to perform the text search in your application.
3. Handle the `sysAppLaunchCmdGoto` launch code, to display the selected result of the text search.

These three steps are described in the following sections.

Handling the `sysAppLaunchCmdSaveData` Launch Code

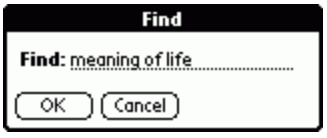
If your application is running when the user taps the **Find** icon , then Palm OS will send the `sysAppLaunchCmdSaveData` launch code to give your application a chance to save any data.

To handle `sysAppLaunchCmdSaveData`, insert this code in your PalmMain function:

```
    case sysAppLaunchCmdSaveData:
                //FrmSaveAllForms ();
                break;
```

To save data when receiving `sysAppLaunchCmdSaveData`, you would uncomment the line `FrmSaveAllForms ();` and implement code that saves the appropriate data.

Handling the `sysAppLaunchCmdFind` Launch Code



Palm OS will send the `sysAppLaunchCmdFind` launch code to allow your application to search for the specified text string. To handle this launch code, you first need to add this code in your PalmMain function:

```
// Launch code sent by global find (text search).
        case sysAppLaunchCmdFind:
                Search((FindParamsPtr)cmdPBP);
                break;
```

Next, you need to implement the `Search` function.

- Call `DmGetNextDatabaseByTypeCreator()` and `DmOpenDatabase()` to

open the application's database(s) for searching.

- Call `FindDrawHeader()` to draw an appropriate header string in the Find results dialog to differentiate matching records.
- Search through the application's records for matching text using the `FindStrInStr()` function and other Data Manager APIs.
- For each successful match, call `FindSaveMatch()` to let the system update its own internal data about the progress of the global find operation. Also, call `FindGetLineBounds()` to retrieve the screen location of the next line in the Find results dialog and draw an appropriate string in the dialog to identify the record that was found.
- Call `DmCloseDatabase()` to close the application's database.

The following code shows an implementation of the `Search` function:

```
static void Search(FindParamsPtr findParams)
{
    Err                     err;
    UInt16                  cardNo = 0;
    UInt16                  fieldNum = 0;
    UInt16                  recordNum;
    Char*                   header;
    Boolean                 done;
    Boolean                 match;
    LocalID                 dbID;
    MemHandle               h;
    DmOpenRef               dbP;
    RectangleType           r;
    DmSearchStateType       searchInfo;
    UInt16                  matchPos;
    UInt16                  matchLength = 0;

    // Find the application's data file.
    err =  DmGetNextDatabaseByTypeCreator (true, &searchInfo, 'DATA',
                              appFileCreator, true, &cardNo, &dbID);
    if (err)
    {
            findParams->more = false;
            return;
    }

    // Display the heading line.
    header = "DataMgrTestDB";
    done = FindDrawHeader(findParams, header);

    if (done) return;

    // Open the database.
    dbP = DmOpenDatabase(cardNo, dbID, findParams->dbAccesMode);
    if (!dbP)
    {
            findParams->more = false;
            return;
    }

    // Search fields for the "find" string.
    recordNum = findParams->recordNum;
    while (true)
    {
```

```
// Because applications can take a long time to finish a find
// when the result may be on the screen or for other reasons,
// users like to be able to stop the find.  Stop the find if an
// event is pending. This stops if the user does something with
// the device.  Because this call slows down the search we perform
// it every so many records instead of every record.  The response
// time should still be short without introducing much extra work
// to the search.

// Note that in the implementation below, if the next 16th record
// is secret the check doesn't happen.  Generally this shouldn't
// be a problem since if most of the records are secret then the
// search won't take long anyways!
    if ((recordNum & 0x000f) == 0 && // every 16th record
            EvtSysEventAvail(true))
    {
            // Stop the search process.
            findParams->more = true;
            break;
    }

    h = DmQueryNextInCategory (dbP,
                               &recordNum,
                               dmAllCategories);

    //Stop searching if there are no more records.
    if (!h){
            findParams->more =false;
            break;
    }


    // get the handle to the record and set busy bit
    // DmQueryRecord() does not mark the busy bit of records.
    h = DmGetRecord(dbP, recordNum);
    if (h) { // could fail due to out of memory!
            DBRecordType * p = (DBRecordType *) MemHandleLock(h);

            match = false;
            if (p->FirstName &&
                    FindStrInStr(p->FirstName,
                                 findParams->strToFind,
                                 &matchPos))
                    match = true;
            else if (p->LastName &&
                    FindStrInStr(p->LastName,
                                 findParams->strToFind,
                                 &matchPos))
                    match = true;
            else if (p->PhoneNum &&
                    FindStrInStr(p->PhoneNum,
                                 findParams->strToFind,
                                 &matchPos))
                    match = true;

            DmReleaseRecord(dbP, recordNum, true);
    }

    if (match)
    {
```

```
            // Add the match to the find paramter block,  if there is no
            // room to display the match the following function will
            // return true.
            done = FindSaveMatch (findParams, recordNum, matchPos,
                fieldNum,matchLength, cardNo, dbID);
            if (done)
            {
                    MemHandleUnlock (h);
                    break;
            }

            // Get the bounds of the region where we
            // will draw the results.
            FindGetLineBounds (findParams, &r);

            // Display the match
            WinDrawTruncChars (findParams->strAsTyped,
                               StrLen(findParams->strAsTyped),
                               r.topLeft.x+1,
                               r.topLeft.y,
                               50);

            findParams->lineNumber++;
            }

            MemHandleUnlock (h);
            recordNum++;
    }

Exit:
DmCloseDatabase (dbP);
}
```

Handling the `sysAppLaunchCmdGoto` Launch Code

Palm OS sends the `sysAppLaunchCmdGoto` launch code when the user taps an entry in the global find results dialog. Your application should respond by opening to the record that corresponds with the tapped entry.

To handle the `sysAppLaunchCmdGoto` launch code, insert this code in your PalmMain function:

```
        // This action code might be sent to the app when
        // it's already running
        // if the use hits the "Go To" button in the
        // Find Results dialog.
        case sysAppLaunchCmdGoTo:
                launched = launchFlags & sysAppLaunchFlagNewGlobals;
                if (launched)
                {
                        error = AppStart();
                        if (error) return (error);
                        FrmGotoForm(MainForm);

                        AppEventLoop ();
```

```
                AppStop ();
        }
```

Sample Application

The attached sample application starts from the data manager sample and enables global find for that sample application.

Look for the "Code Added" comments in Sample applications:

- Windows
- Mac